

Calgary_Crime_Data_Analysis_and_Neural_Network_Prediction

June 23, 2025

1 Calgary Crime Data Analysis and Neural Network Prediction

The aim of this project is to use the Crime and Disorder Data provided by the City of Calgary's data website to analyze the data and predict the number of crimes that will occur in the future. The data is from 2018 to 2024 and contains the number of crimes that occurred in Calgary for each month. After thoroughly analyzing the data, I will be building a neural network model and optimizing it to predict the number of crimes that will occur in the future.

1.1 Data Dictionary

Column Name	Description
Community Name	The name of the community in Calgary
Category	The type of crime that occurred
Crime Count	The number of crimes that occurred in that month
Year	The year the crime occurred
Month	The month the crime occurred

1.2 Strategy

1. Loading the data and understanding the data
2. Data Preprocessing - cleaning the data and preparing it for analysis
3. Exploratory Data Analysis - Analyzing the data to understand the trends and patterns
4. Building a Neural Network Model
5. Optimizing the model
6. Training the model
7. Predicting the number of crimes that will occur in the future

```
[1]: #Importing the Libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

```
[4]: #loading the dataset
df = pd.read_csv('Community_Crime_Statistics_20240522.csv')
df.head()
```

```
[4]:
```

	Community	Category	Crime Count	Year	Month
0	01B	Assault (Non-domestic)	1	2022	11
1	01B	Break & Enter - Commercial	1	2019	6
2	01B	Break & Enter - Commercial	1	2019	8
3	01B	Break & Enter - Commercial	2	2020	3
4	01B	Break & Enter - Commercial	2	2020	7

Here the is the representation of first 5 records of the data, which gives a brief informaton about the data. Since the dataset is alphabetically sorted by the community name, the data is not in a chronological order.

1.2.1 Data Preprocessing

```
[5]: #shape of the dataset
df.shape
```

```
[5]: (70661, 5)
```

Here we have bearly 70661 records and 5 columns. Therefore, we have enough data for preparing an analysis and developing a model for prediction.

```
[6]: #checking for missing values
df.isnull().sum()
```

```
[6]: Community      0
      Category      0
      Crime Count   0
      Year          0
      Month         0
      dtype: int64
```

The dataset is pretty clean and does not have any missing values.

```
[7]: #checking for the datatypes
df.dtypes
```

```
[7]: Community      object
      Category      object
      Crime Count   int64
      Year          int64
      Month         int64
      dtype: object
```

Making sure that the columns have correct datatype, before I proceed with the analysis.

```
[8]: #Descriptive statistics
df.describe()
```

```
[8]:
```

	Crime Count	Year	Month
count	70661.000000	70661.000000	70661.000000
mean	2.855748	2020.618616	6.369242
std	3.664965	1.825330	3.451445
min	1.000000	2018.000000	1.000000
25%	1.000000	2019.000000	3.000000
50%	2.000000	2021.000000	6.000000
75%	3.000000	2022.000000	9.000000
max	111.000000	2024.000000	12.000000

1.3 Exploratory Data Analysis

In the exploratory data analysis, I will be analyzing the data to understand the trends and patterns in the data. Through this analysis, I will be able to understand the data better and build a better model for prediction.

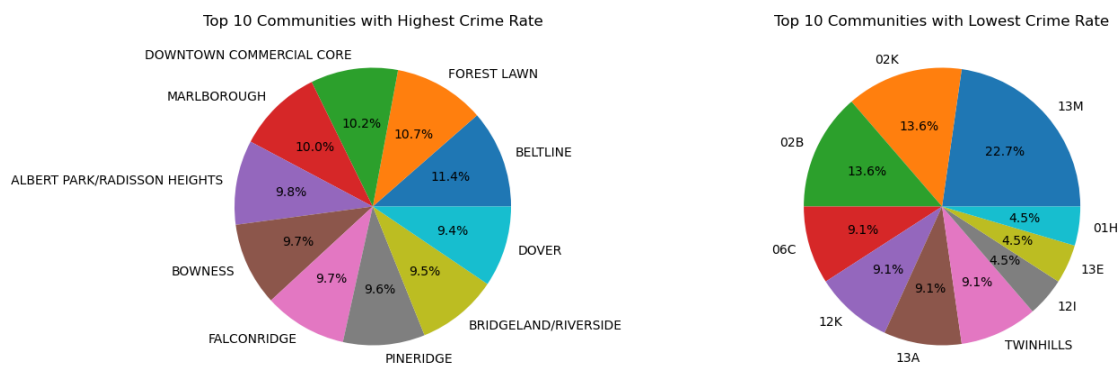
1.3.1 Community Distribution

```
[9]: fig, ax = plt.subplots(1, 2, figsize=(15, 5))

#Top 10 Communities with Highest Crime Rate
df['Community'].value_counts().head(10).plot.pie(autopct='%1.1f%%', ax = ax[0])
ax[0].set_title('Top 10 Communities with Highest Crime Rate')
ax[0].set_ylabel('')

#Top 10 Communities with Lowest Crime Rate
df['Community'].value_counts().tail(10).plot.pie(autopct='%1.1f%%', ax = ax[1])
ax[1].set_title('Top 10 Communities with Lowest Crime Rate')
ax[1].set_ylabel('')
```

```
[9]: Text(0, 0.5, '')
```



These pie charts show the distribution of crimes in each community. The first pie chart shows the top 10 most dangerous communities in Calgary. The second pie chart shows the distribution of top

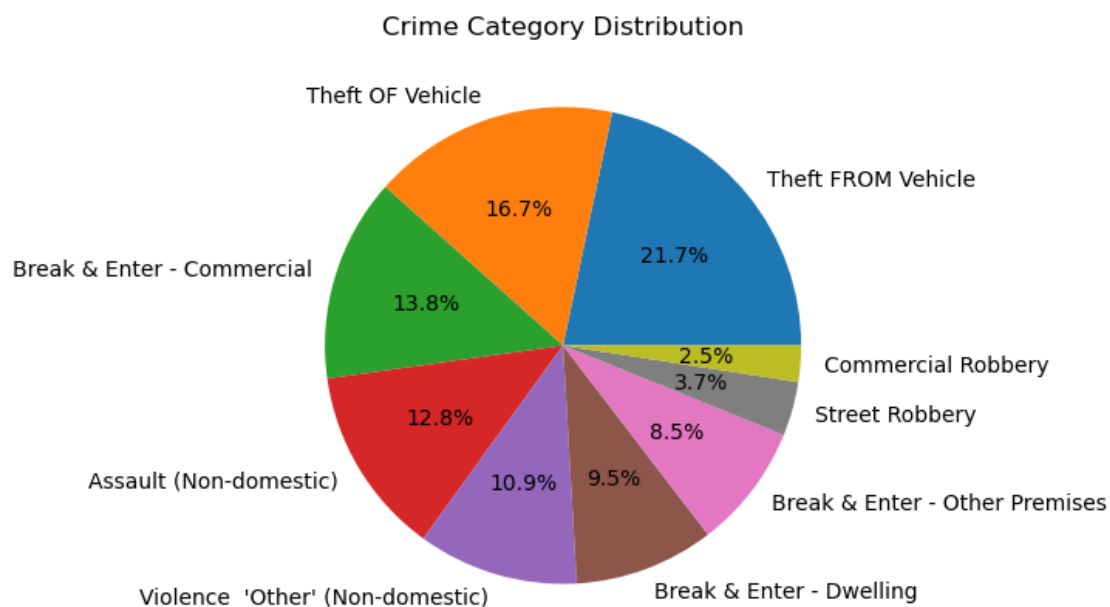
10 safest communities in Calgary. In the first pie chart, Beltline is the most dangerous community in Calgary with 11.4% of the top crimes in number, followed by Forest Lawn with 10.7% and Downtown Commercial Core with 10.2%. In the second pie chart, the safest community is 13M with 22.7% of the least crimes in number, followed by 02K with 13.6% and 02B with 13.6%.

This is note that all these observations are without any bias and completely based on the data from the city of Calgary website.

1.3.2 Crime Category Distribution

```
[10]: plt.figure(figsize=(5, 5))
      df['Category'].value_counts().plot.pie(autopct='%1.1f%%')
      plt.title('Crime Category Distribution')
      plt.ylabel('')
```

```
[10]: Text(0, 0.5, '')
```

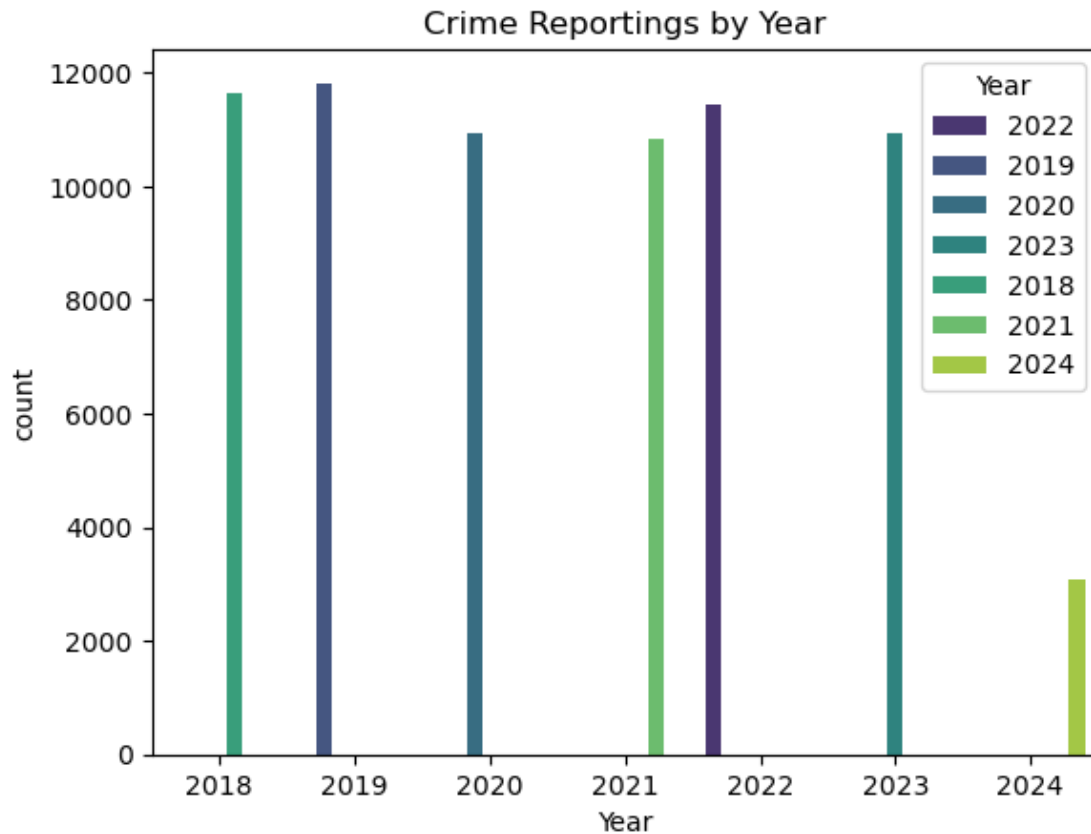


This graph shows the distribution of crimes in each category by the number of crimes. The top crime category is Theft from Vehicle with 21.7% of the total crimes, followed by Theft of Vehicle with 16.7% and Break and Enter - Commercial with 13.8%. The least crime category includes commercial or street robbery.

1.3.3 Crime Reportings Over the Years

```
[13]: sns.countplot(x='Year', data=df, hue=df['Year'].astype(str), palette='viridis').  
      ↪set_title('Crime Reportings by Year')
```

```
[13]: Text(0.5, 1.0, 'Crime Reportings by Year')
```

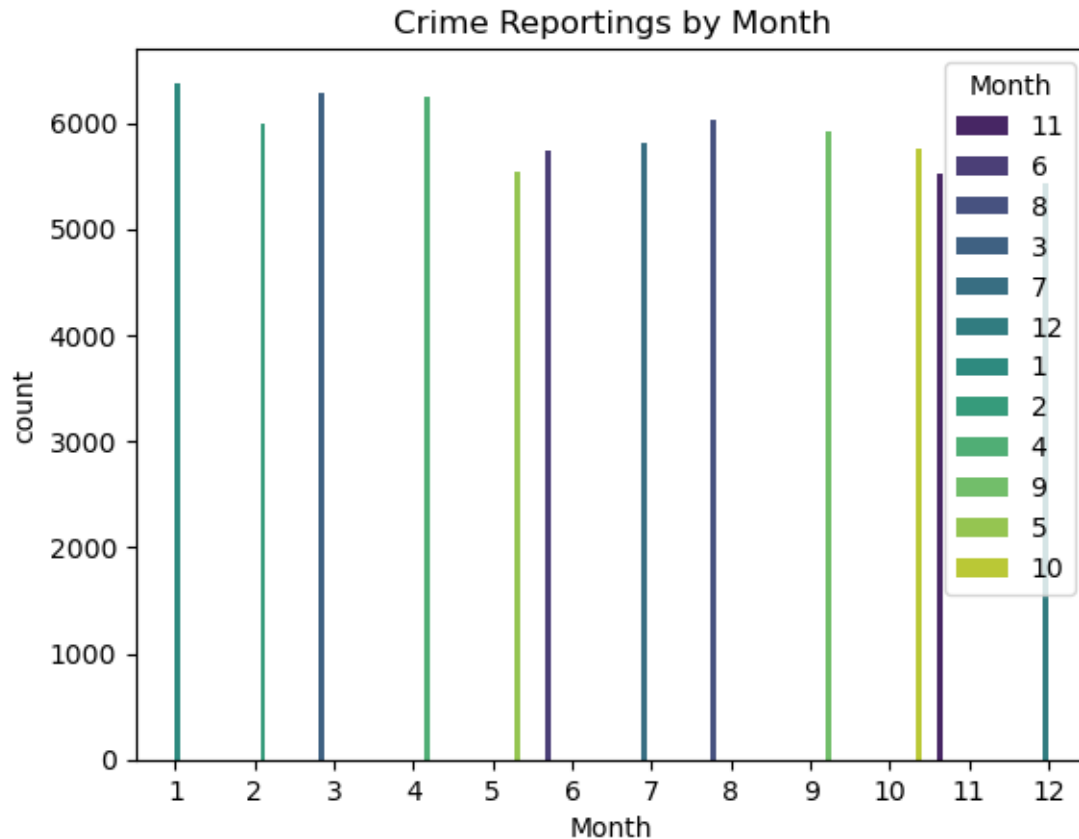


This bar graph shows the distribution of number of crimes reported in the year. The year 2019 had the highest reportings of crimes followed by 2022 and 2018. The crime reportings in 2024 are less due to limited data till April 2024.

1.3.4 Crime Reportings by Month

```
[15]: sns.countplot(x='Month', data=df, hue=df['Month'].astype(str),  
      ↪palette='viridis').set_title('Crime Reportings by Month')
```

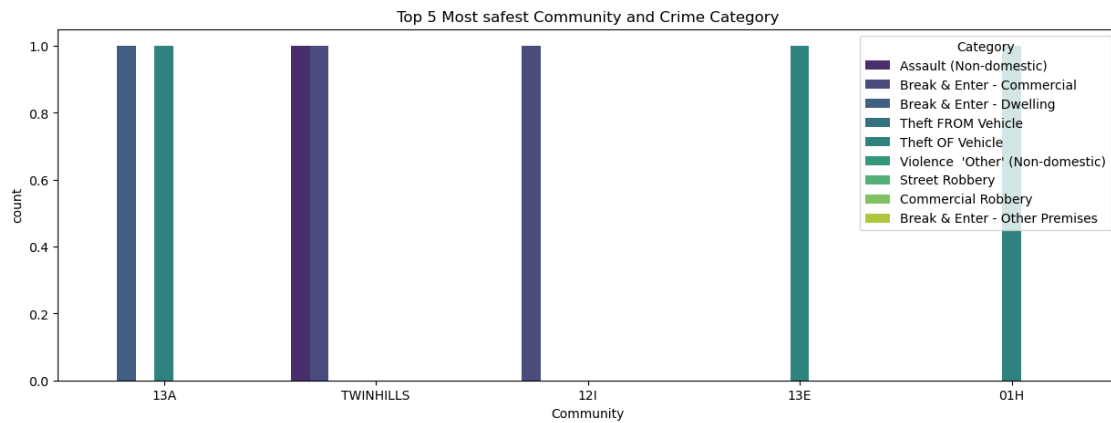
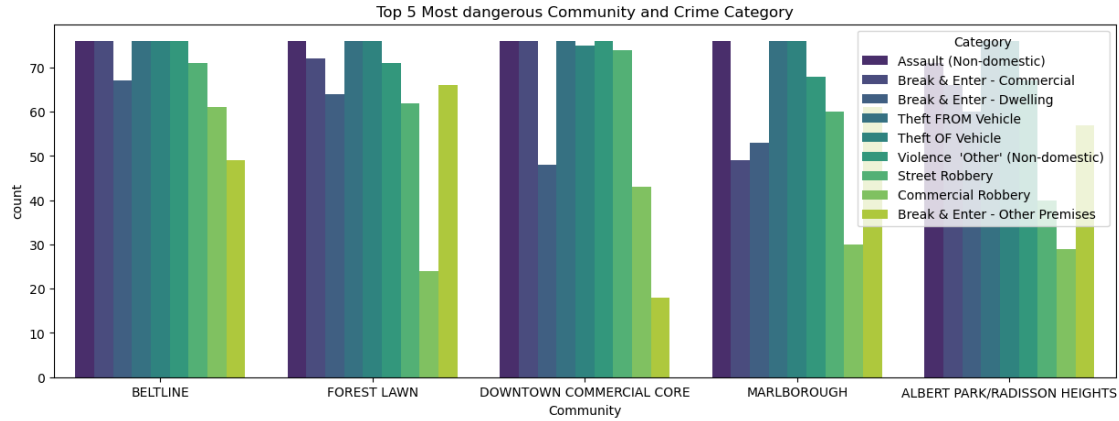
```
[15]: Text(0.5, 1.0, 'Crime Reportings by Month')
```



1.3.5 Community and Category Analysis

```
[16]: plt.figure(figsize=(15, 5))
sns.countplot(x = 'Community', data = df, hue = 'Category', palette='viridis',
order = df['Community'].value_counts().head(5).index).set_title('Top 5 Most
dangerous Community and Crime Category')
sns.move_legend(plt.gca(), "upper right")
plt.figure(figsize=(15, 5))
sns.countplot(x = 'Community', data = df, hue = 'Category', palette='viridis',
order = df['Community'].value_counts().tail(5).index).set_title('Top 5 Most
safest Community and Crime Category')
```

```
[16]: Text(0.5, 1.0, 'Top 5 Most safest Community and Crime Category')
```

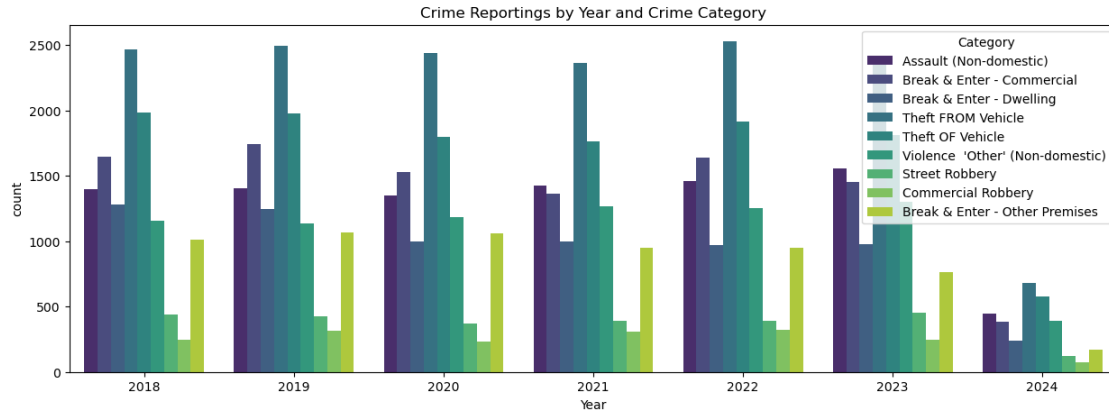


These two graphs shows the analysis of communities with the crime category. This help us to visualize the pattern of crime in each community. We can see that certain categories are more common in certain communities than others. In the top 5 dangerous communities, Forest Lawn has the highest of Break & Enter - other premises, Malbrough has the lowest Commerical Robbery. These are the few examples of the analysis.

1.3.6 Year and Category Analysis

```
[17]: plt.figure(figsize=(15, 5))
sns.countplot(x = 'Year', data = df, hue = 'Category', palette='viridis').
      ↪set_title('Crime Reportings by Year and Crime Category')
```

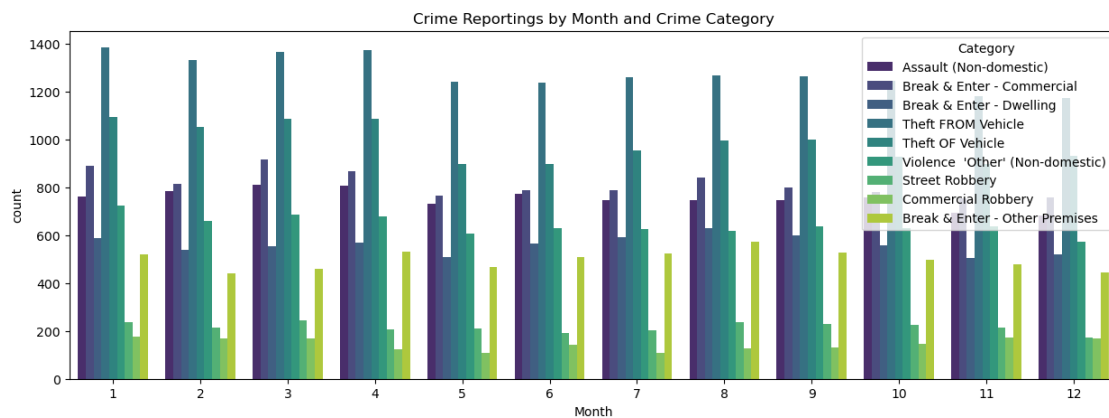
```
[17]: Text(0.5, 1.0, 'Crime Reportings by Year and Crime Category')
```



1.3.7 Month and Category Analysis

```
[18]: plt.figure(figsize=(15, 5))
sns.countplot(x = 'Month', data = df, hue = 'Category', palette='viridis').
      set_title('Crime Reportings by Month and Crime Category')
```

```
[18]: Text(0.5, 1.0, 'Crime Reportings by Month and Crime Category')
```



From the above, graphs, charts, and visualization I have studied the patterns, trends and relationships in the data. This will help me to build a better model for prediction.

1.4 Data Preprocessing Part 2

```
[19]: from sklearn.preprocessing import LabelEncoder

#Label Encoding Object
le = LabelEncoder()
```



```

#Object type columns
object_type_columns = df.select_dtypes(include='object').columns

#Label Encoding
for col in object_type_columns:
    df[col] = le.fit_transform(df[col])
df.head()

```

```

[19]:
Community  Category  Crime Count  Year  Month
0         0         0           1  2022    11
1         0         1           1  2019     6
2         0         1           1  2019     8
3         0         1           2  2020     3
4         0         1           2  2020     7

```

1.5 Building a Neural Network Model

```

[20]: # Prepare sequences for LSTM
def create_sequences(data, seq_length):
    xs = []
    ys = []
    for i in range(len(data) - seq_length):
        x = data.iloc[i:(i + seq_length)].to_numpy()
        y = data.iloc[i + seq_length]['Crime Count']
        xs.append(x)
        ys.append(y)
    return np.array(xs), np.array(ys)

```

```

[22]: seq_length = 3
X, y = create_sequences(df, seq_length)

```

1.5.1 Train Test Split

```

[23]: from sklearn.model_selection import train_test_split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3,
↪random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
↪random_state=42)

```

1.5.2 Building and Training the LSTM Model

```

[24]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam

```

```
[25]: # Build the LSTM model
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(seq_length, X_train.
↳shape[2])))
model.add(Dropout(0.2))
model.add(Dense(1))

# Compile the model
optimizer = Adam(learning_rate=0.001)
model.compile(optimizer=optimizer, loss='mse')

# Train the model
history = model.fit(X_train, y_train, epochs=100, validation_data=(X_val,
↳y_val), batch_size=16)
```

C:\Users\jeane\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:204:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(**kwargs)
```

```
Epoch 1/100
3092/3092          22s 5ms/step -
loss: 251.5637 - val_loss: 12.2297
Epoch 2/100
3092/3092          20s 5ms/step -
loss: 13.5650 - val_loss: 5.8945
Epoch 3/100
3092/3092          19s 5ms/step -
loss: 6.9696 - val_loss: 5.9602
Epoch 4/100
3092/3092          21s 5ms/step -
loss: 6.2019 - val_loss: 4.7969
Epoch 5/100
3092/3092          21s 5ms/step -
loss: 5.8411 - val_loss: 4.8804
Epoch 6/100
3092/3092          21s 5ms/step -
loss: 6.1632 - val_loss: 5.4062
Epoch 7/100
3092/3092          20s 5ms/step -
loss: 5.7815 - val_loss: 4.7649
Epoch 8/100
3092/3092          21s 5ms/step -
loss: 6.0311 - val_loss: 6.3340
Epoch 9/100
3092/3092          20s 5ms/step -
loss: 5.8542 - val_loss: 6.3976
```

Epoch 10/100
3092/3092 21s 5ms/step -
loss: 5.7057 - val_loss: 5.0454
Epoch 11/100
3092/3092 20s 5ms/step -
loss: 5.7352 - val_loss: 5.0795
Epoch 12/100
3092/3092 21s 5ms/step -
loss: 5.9081 - val_loss: 4.7678
Epoch 13/100
3092/3092 16s 5ms/step -
loss: 6.1661 - val_loss: 5.1894
Epoch 14/100
3092/3092 21s 5ms/step -
loss: 6.0288 - val_loss: 4.8561
Epoch 15/100
3092/3092 19s 5ms/step -
loss: 5.9553 - val_loss: 4.8995
Epoch 16/100
3092/3092 22s 5ms/step -
loss: 6.3011 - val_loss: 5.0411
Epoch 17/100
3092/3092 20s 5ms/step -
loss: 6.0121 - val_loss: 5.7098
Epoch 18/100
3092/3092 20s 5ms/step -
loss: 5.5410 - val_loss: 4.8678
Epoch 19/100
3092/3092 21s 5ms/step -
loss: 6.1498 - val_loss: 4.8413
Epoch 20/100
3092/3092 15s 5ms/step -
loss: 6.0787 - val_loss: 4.8475
Epoch 21/100
3092/3092 20s 5ms/step -
loss: 5.9668 - val_loss: 4.9760
Epoch 22/100
3092/3092 15s 5ms/step -
loss: 5.4583 - val_loss: 5.0750
Epoch 23/100
3092/3092 20s 5ms/step -
loss: 5.9803 - val_loss: 5.0467
Epoch 24/100
3092/3092 21s 5ms/step -
loss: 6.0039 - val_loss: 5.0255
Epoch 25/100
3092/3092 21s 5ms/step -
loss: 5.5836 - val_loss: 4.8262

Epoch 26/100
 3092/3092 21s 5ms/step -
 loss: 6.0497 - val_loss: 5.2300
 Epoch 27/100
 3092/3092 19s 5ms/step -
 loss: 5.9851 - val_loss: 4.9157
 Epoch 28/100
 3092/3092 15s 5ms/step -
 loss: 6.2704 - val_loss: 5.3035
 Epoch 29/100
 3092/3092 21s 5ms/step -
 loss: 5.3066 - val_loss: 5.3559
 Epoch 30/100
 3092/3092 20s 5ms/step -
 loss: 6.0417 - val_loss: 4.8725
 Epoch 31/100
 3092/3092 20s 5ms/step -
 loss: 5.7135 - val_loss: 5.0966
 Epoch 32/100
 3092/3092 230s 72ms/step
 - loss: 6.3261 - val_loss: 4.8162
 Epoch 33/100
 3092/3092 20s 4ms/step -
 loss: 5.8718 - val_loss: 4.8496
 Epoch 34/100
 3092/3092 19s 4ms/step -
 loss: 5.9141 - val_loss: 6.3403
 Epoch 35/100
 3092/3092 22s 4ms/step -
 loss: 5.7014 - val_loss: 5.1423
 Epoch 36/100
 3092/3092 12s 4ms/step -
 loss: 5.8030 - val_loss: 5.0223
 Epoch 37/100
 3092/3092 12s 4ms/step -
 loss: 5.8917 - val_loss: 5.1459
 Epoch 38/100
 3092/3092 13s 4ms/step -
 loss: 5.5843 - val_loss: 4.8093
 Epoch 39/100
 3092/3092 21s 4ms/step -
 loss: 5.7538 - val_loss: 4.8305
 Epoch 40/100
 3092/3092 22s 5ms/step -
 loss: 5.9014 - val_loss: 5.3673
 Epoch 41/100
 3092/3092 13s 4ms/step -
 loss: 5.8366 - val_loss: 5.0571

Epoch 42/100
 3092/3092 21s 4ms/step -
 loss: 5.6996 - val_loss: 4.7567
 Epoch 43/100
 3092/3092 15s 5ms/step -
 loss: 6.0073 - val_loss: 4.6984
 Epoch 44/100
 3092/3092 13s 4ms/step -
 loss: 6.0858 - val_loss: 5.0452
 Epoch 45/100
 3092/3092 22s 5ms/step -
 loss: 5.5833 - val_loss: 4.9472
 Epoch 46/100
 3092/3092 20s 5ms/step -
 loss: 5.6051 - val_loss: 5.0778
 Epoch 47/100
 3092/3092 21s 5ms/step -
 loss: 5.8960 - val_loss: 4.9867
 Epoch 48/100
 3092/3092 21s 5ms/step -
 loss: 5.4936 - val_loss: 4.9184
 Epoch 49/100
 3092/3092 21s 5ms/step -
 loss: 5.6967 - val_loss: 4.7912
 Epoch 50/100
 3092/3092 21s 5ms/step -
 loss: 6.0719 - val_loss: 4.8772
 Epoch 51/100
 3092/3092 21s 5ms/step -
 loss: 5.4043 - val_loss: 5.5645
 Epoch 52/100
 3092/3092 20s 5ms/step -
 loss: 5.3534 - val_loss: 5.0747
 Epoch 53/100
 3092/3092 13s 4ms/step -
 loss: 5.6277 - val_loss: 4.8563
 Epoch 54/100
 3092/3092 14s 5ms/step -
 loss: 5.7870 - val_loss: 4.7994
 Epoch 55/100
 3092/3092 20s 4ms/step -
 loss: 5.3661 - val_loss: 4.9158
 Epoch 56/100
 3092/3092 21s 5ms/step -
 loss: 5.8422 - val_loss: 6.0016
 Epoch 57/100
 3092/3092 14s 4ms/step -
 loss: 5.6656 - val_loss: 4.8747

Epoch 58/100
 3092/3092 13s 4ms/step -
 loss: 6.1306 - val_loss: 5.6654
 Epoch 59/100
 3092/3092 13s 4ms/step -
 loss: 6.0290 - val_loss: 5.1045
 Epoch 60/100
 3092/3092 13s 4ms/step -
 loss: 6.5756 - val_loss: 4.9150
 Epoch 61/100
 3092/3092 13s 4ms/step -
 loss: 5.7425 - val_loss: 4.8454
 Epoch 62/100
 3092/3092 13s 4ms/step -
 loss: 6.0390 - val_loss: 4.8389
 Epoch 63/100
 3092/3092 13s 4ms/step -
 loss: 5.6260 - val_loss: 5.7533
 Epoch 64/100
 3092/3092 14s 5ms/step -
 loss: 5.5923 - val_loss: 4.8609
 Epoch 65/100
 3092/3092 14s 5ms/step -
 loss: 5.6293 - val_loss: 4.7365
 Epoch 66/100
 3092/3092 19s 4ms/step -
 loss: 5.5847 - val_loss: 5.3330
 Epoch 67/100
 3092/3092 20s 4ms/step -
 loss: 6.5425 - val_loss: 5.0413
 Epoch 68/100
 3092/3092 22s 5ms/step -
 loss: 6.1946 - val_loss: 4.7477
 Epoch 69/100
 3092/3092 19s 4ms/step -
 loss: 5.6266 - val_loss: 5.3138
 Epoch 70/100
 3092/3092 21s 4ms/step -
 loss: 5.6248 - val_loss: 5.0164
 Epoch 71/100
 3092/3092 13s 4ms/step -
 loss: 6.0090 - val_loss: 4.8009
 Epoch 72/100
 3092/3092 13s 4ms/step -
 loss: 6.1185 - val_loss: 4.8250
 Epoch 73/100
 3092/3092 13s 4ms/step -
 loss: 6.3594 - val_loss: 5.2589

Epoch 74/100
3092/3092 13s 4ms/step -
loss: 5.7693 - val_loss: 5.0518
Epoch 75/100
3092/3092 13s 4ms/step -
loss: 6.2123 - val_loss: 4.9157
Epoch 76/100
3092/3092 13s 4ms/step -
loss: 5.6915 - val_loss: 5.2201
Epoch 77/100
3092/3092 13s 4ms/step -
loss: 5.5597 - val_loss: 5.2880
Epoch 78/100
3092/3092 13s 4ms/step -
loss: 5.7704 - val_loss: 5.0844
Epoch 79/100
3092/3092 13s 4ms/step -
loss: 6.0445 - val_loss: 4.8421
Epoch 80/100
3092/3092 13s 4ms/step -
loss: 5.5510 - val_loss: 5.1420
Epoch 81/100
3092/3092 13s 4ms/step -
loss: 5.9810 - val_loss: 4.7156
Epoch 82/100
3092/3092 13s 4ms/step -
loss: 5.7139 - val_loss: 4.7140
Epoch 83/100
3092/3092 13s 4ms/step -
loss: 5.7617 - val_loss: 4.7454
Epoch 84/100
3092/3092 21s 4ms/step -
loss: 5.8934 - val_loss: 4.6678
Epoch 85/100
3092/3092 13s 4ms/step -
loss: 5.7133 - val_loss: 4.6700
Epoch 86/100
3092/3092 13s 4ms/step -
loss: 5.7143 - val_loss: 4.9402
Epoch 87/100
3092/3092 13s 4ms/step -
loss: 5.7124 - val_loss: 4.6911
Epoch 88/100
3092/3092 13s 4ms/step -
loss: 5.5668 - val_loss: 4.7203
Epoch 89/100
3092/3092 13s 4ms/step -
loss: 5.8470 - val_loss: 4.7443

```

Epoch 90/100
3092/3092          13s 4ms/step -
loss: 5.5785 - val_loss: 5.1262
Epoch 91/100
3092/3092          13s 4ms/step -
loss: 5.9809 - val_loss: 5.4849
Epoch 92/100
3092/3092          13s 4ms/step -
loss: 5.6021 - val_loss: 4.7013
Epoch 93/100
3092/3092          13s 4ms/step -
loss: 5.7492 - val_loss: 4.9992
Epoch 94/100
3092/3092          13s 4ms/step -
loss: 6.3697 - val_loss: 4.7085
Epoch 95/100
3092/3092          13s 4ms/step -
loss: 5.4280 - val_loss: 5.3089
Epoch 96/100
3092/3092          13s 4ms/step -
loss: 6.2963 - val_loss: 4.9308
Epoch 97/100
3092/3092          13s 4ms/step -
loss: 5.5762 - val_loss: 4.6679
Epoch 98/100
3092/3092          20s 4ms/step -
loss: 5.7880 - val_loss: 5.0532
Epoch 99/100
3092/3092          13s 4ms/step -
loss: 5.6257 - val_loss: 5.2319
Epoch 100/100
3092/3092          13s 4ms/step -
loss: 5.6620 - val_loss: 5.2158

```

```

[26]: plt.figure(figsize=(12, 6))
      plt.plot(history.history['loss'], label='Training Loss')
      plt.plot(history.history['val_loss'], label='Validation Loss')
      plt.title('Training and Validation Loss Over Epochs')
      plt.xlabel('Epochs')
      plt.ylabel('Loss')
      plt.legend()
      plt.grid(True)
      plt.show()

```




```
[27]: # Evaluate the model
test_loss = model.evaluate(X_test, y_test)
print(f'Test Loss: {test_loss}')

# Predictions
y_pred = model.predict(X_test)

print(f'Predictions: {y_pred.flatten()}')
print(f'True Values: {y_test.flatten()}')
```

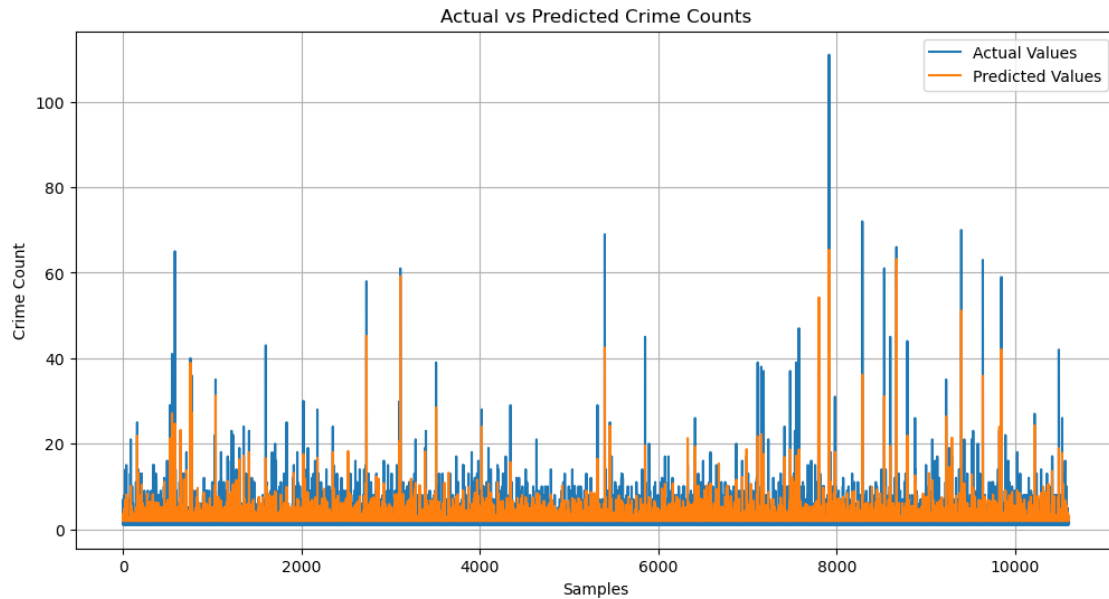
```
332/332          1s 3ms/step -
loss: 5.0991
Test Loss: 5.378209114074707
332/332          2s 4ms/step
Predictions: [3.3283691 1.9982157 2.1404734 ... 3.3390732 1.9471747 2.8811   ]
True Values: [2 1 1 ... 1 2 2]
```

1.6 Model Evaluation

1.6.1 Actual vs Predicted Values

```
[28]: # Plotting Actual vs Predicted Values
plt.figure(figsize=(12, 6))
plt.plot(y_test, label='Actual Values')
plt.plot(y_pred, label='Predicted Values')
plt.title('Actual vs Predicted Crime Counts')
plt.xlabel('Samples')
```

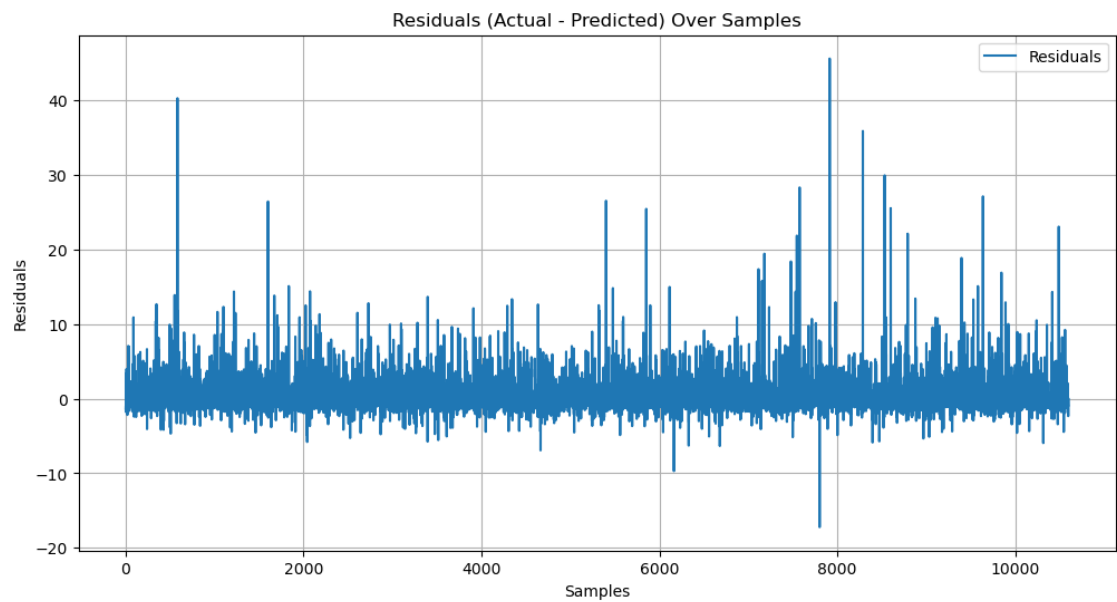
```
plt.ylabel('Crime Count')
plt.legend()
plt.grid(True)
plt.show()
```



1.6.2 Residual Plot

```
[29]: # Calculating residuals
residuals = y_test.flatten() - y_pred.flatten()

# Plotting residuals
plt.figure(figsize=(12, 6))
plt.plot(residuals, label='Residuals')
plt.title('Residuals (Actual - Predicted) Over Samples')
plt.xlabel('Samples')
plt.ylabel('Residuals')
plt.legend()
plt.grid(True)
plt.show()
```



[]: